Role-Based Access Control (RBAC) System for a Hospital Database: A Comparative Study of Application-Level and Database-Level Security

Project Title: Hospital RBAC Database Security System — CityCare Hospital Management System

Name : Deepaguntla Sohar Baba

Reg.No : 235805394

Course Coordinator: Dr. Prakash B Metre

Course: Database and Application Security (CSE_4092)

Abstract

Modern healthcare and enterprise information systems store highly sensitive records — medical histories, billing information, and personal identifiers — that must be accessible only to authorized personnel. In practice, many systems implement access control exclusively within the application layer, using conditional logic to check a user's role before permitting an action. This approach is fragile: a single overlooked check, a forgotten conditional, or a successful SQL injection attack can expose the entire dataset regardless of how carefully the role logic was designed elsewhere in the system.

This project designs, implements, and evaluates two parallel versions of a hospital records management system — "CityCare Hospital" — to demonstrate this problem concretely and to show its resolution. The first version, the **insecure baseline**, enforces all access control in Python application code while every request reaches the database through a single, fully-privileged superuser connection. The second version, the **secure implementation**, enforces access control natively inside PostgreSQL itself, using dedicated login roles per user type, explicit privilege grants and revocations, connection limits analogous to Oracle profiles, and Row-Level Security (RLS) policies that restrict visibility to individual rows based on the authenticated user's identity.

Two classes of real vulnerabilities were deliberately introduced into the insecure baseline: an Insecure Direct Object Reference (IDOR) permitting any authenticated user to view any patient's medical record by manipulating a URL parameter, and a SQL Injection vulnerability in a patient search feature, exploited first to bypass a search filter and subsequently to perform a UNION-based attack extracting confidential billing data through an unrelated search form. Both attacks succeeded fully against the insecure baseline. When the identical attacks were repeated against the secure implementation — using the same user accounts, the same URLs, and the same payloads — both were blocked entirely at the database layer, and the attempts were recorded in an audit log. This confirms the central thesis of the project: database-level access control provides an independent, enforceable layer of defense that holds even when application-level logic is flawed or bypassed.

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

Manipal Institute of Technology, Bengaluru

CERTIFICATE

This is to certify that the course project titled "Role-Based Access Control (RBAC) System for a Hospital Database: A Comparative Study of Application-Level and Database-Level Security" is a record of the bona fide work carried out by Deepaguntla Sohar Baba (Reg. No. 235805394) in partial fulfilment of the requirements of the course Database and Application Security (CSE_4092), forming part of the award of the Degree of Bachelor of Technology (B.Tech.) in Computer Science and Engineering (Core) of Manipal Institute of Technology, Bengaluru, Karnataka (a constituent institute of Manipal Academy of Higher Education), during the academic year 2026–2027.

Dr. Prakash B Metre

Course Coordinator,

Database and Application Security (CSE_4092)

SOCE, M.I.T, Bengaluru

ACKNOWLEDGEMENTS

I express my sincere gratitude to the School of Computer Engineering, Manipal Institute of Technology, Bengaluru, for providing me the opportunity to undertake this project as part of the course *Database and Application Security (CSE_4092)*, and for encouraging hands-on, industry-aligned learning in the area of data protection and secure system design.

I am particularly grateful to my Course Coordinator, Dr. Prakash B Metre, for his valuable guidance, encouragement, and support throughout the course and during the preparation of this project and report. His teaching on user administration, privileges, roles, and database auditing formed the foundation of this work.

I also acknowledge the authors and maintainers of the PostgreSQL documentation, the OWASP project, and the other learning resources referenced in this report. Their material was invaluable in understanding Role-Based Access Control, Row-Level Security, and common web application vulnerabilities such as Insecure Direct Object Reference and SQL Injection. Finally, I thank my family and friends for their continuous motivation and support.

1. Introduction

1.1 Background

Healthcare information systems are among the most sensitive categories of software in active use today. A hospital database typically stores a patient's personal identifiers, contact details, diagnosis history, treatment records, and billing information — all of which carry legal and ethical obligations around confidentiality. In most jurisdictions, unauthorized disclosure of medical records is treated as a serious breach, both legally and in terms of patient trust.

Despite this sensitivity, many student projects, prototype systems, and even some production systems implement access control entirely within the application layer. A typical implementation checks the logged-in user's role in a conditional statement before executing a database query or rendering a page — for example, `if user.role == "doctor": show_record()`. While this is a reasonable first layer of defense, it suffers from a structural weakness: the database itself places no restriction on what the connected application can request. If the connecting account has full read/write privileges on every table (as is common when a single service account is used for the entire application), then any bug, omission, or successful attack against the application layer grants the attacker the same unrestricted access that the legitimate application enjoys.

1.2 Problem Statement

Sensitive hospital records — particularly medical diagnoses and billing information — are frequently over-exposed in systems where access control is implemented solely at the application layer rather than enforced natively at the database layer. A single missing authorization check on one route, or a successful SQL injection attack, can expose data that should be restricted to specific roles (e.g., treating physicians, billing staff), regardless of how correctly access control is implemented elsewhere in the same codebase.

This project investigates whether shifting enforcement into the database itself — using native role-based privilege management and row-level filtering — can provide a meaningfully stronger and more resilient security posture than application-only enforcement, and demonstrates this empirically through controlled attack-and-defense testing on two functionally identical systems.

1.3 Objectives

The objectives of this project are to:

1. Design and implement a realistic hospital records management system covering patients, doctors, appointments, medical records, billing, and an audit trail.
2. Build an **insecure baseline** version in which all access control is implemented in application code, connecting to the database through a single, fully-privileged account.
3. Deliberately introduce and document two realistic, class-representative vulnerabilities into the insecure baseline: an Insecure Direct Object Reference (IDOR) and a SQL Injection vulnerability.
4. Demonstrate successful exploitation of both vulnerabilities against the insecure baseline.

5. Build a **secure implementation** of the same system that enforces access control using PostgreSQL's native role system, the GRANT/REVOKE privilege model, connection-limit profiles, and Row-Level Security policies.
6. Repeat the identical attacks against the secure implementation and document the results.
7. Implement an audit logging mechanism that records denied access attempts at the database layer, independent of the application.
8. Draw conclusions regarding the necessity of database-level enforcement as a complement to application-level access control (defense-in-depth).

1.4 Scope

The project scope covers the core hospital workflow: patient registration and directory information, doctor-patient assignment, appointment scheduling, medical record creation and viewing, and billing. Five application-level user roles are modeled: system administrator, doctor, nurse, receptionist, and billing clerk. A patient-facing portal role is intentionally excluded from the secure implementation's database-role mapping to keep the project focused on staff-side access control, which is where the syllabus's Module 3 content (user administration, profiles, privileges, and roles) is most directly applicable.

1.5 Report Organization

Section 2 reviews relevant background concepts: access control models and the specific vulnerability classes exploited in this project. Section 3 presents the system design, including the entity-relationship model and the role-privilege matrix. Section 4 describes the implementation of both the insecure and secure versions in detail, including the actual SQL used to define roles, privileges, and Row-Level Security policies. Section 5 presents the attacks performed and their results in both versions, including audit log evidence. Section 6 concludes with a discussion of defense-in-depth. Section 7 lists references.

2. Literature / Background

2.1 Access Control Models

Access control models govern how a system decides whether a given subject (user, process) may perform a given action on a given object (table, row, resource). Three models are commonly discussed in database and application security literature:

- **Discretionary Access Control (DAC):** The owner of a resource decides who may access it and grants permissions accordingly. PostgreSQL's `GRANT/REVOKE` privilege model is fundamentally discretionary — a table owner (or a role with sufficient privilege) decides which roles may `SELECT`, `INSERT`, `UPDATE`, or `DELETE` on that table.

- **Mandatory Access Control (MAC):** Access decisions are governed by a central policy based on security labels or classifications (e.g., "Confidential", "Secret") attached to both subjects and objects, and are not discretionary to individual resource owners. MAC is more common in military and government systems and is not the primary model used in this project, though Row-Level Security policies share some conceptual similarity in that they enforce a rule independent of what the connecting application requests.
- **Role-Based Access Control (RBAC):** Permissions are assigned to roles rather than to individual users, and users are assigned to roles. This decouples "who a user is" from "what a user may do," and is the dominant model in enterprise systems because it scales — adding a new doctor only requires assigning them to the existing `doctor_role`, not re-defining permissions from scratch.

PostgreSQL's native `ROLE` system directly implements RBAC: this project defines one PostgreSQL login role per application-level user type (`doctor_role`, `nurse_role`, `receptionist_role`, `billing_role`, `admin_role`), and every user of that type connects to the database using the same role's credentials, rather than having individually-scoped database permissions.

2.2 Defense-in-Depth

Defense-in-depth is the security principle that a system should not rely on a single layer of protection; instead, multiple independent layers should each be capable of stopping an attack, such that the failure of one layer does not result in total compromise. In the context of this project, the application layer (Python route handlers checking `session["role"]`) is one layer, and the database layer (PostgreSQL roles, privileges, and Row-Level Security) is a second, independent layer. The central experiment of this project is to show that when the first layer fails or is bypassed, the second layer still holds.

2.3 Relevant Vulnerability Classes (OWASP Top 10)

Two vulnerability classes from the OWASP Top 10 — an internationally recognized reference list of critical web application security risks — are directly represented in this project's insecure baseline:

- **A01:2021 — Broken Access Control:** This category covers failures where restrictions on what authenticated users are allowed to do are not properly enforced. The Insecure Direct Object Reference (IDOR) implemented in this project is a textbook instance: the application checks that *a* user is logged in, but not *which* user, nor whether that user is entitled to the specific record being requested via its identifier in the URL.
- **A03:2021 — Injection:** This category covers failures where untrusted input is interpreted as part of a command or query. The SQL Injection vulnerability implemented in this project's patient search feature falls directly into this category: user-supplied search text is concatenated directly into a SQL string rather than being passed as a bound parameter, permitting an attacker to alter the structure of the executed query.

2.4 Row-Level Security

Standard SQL privilege management (`GRANT`/`REVOKE`) operates at the level of entire tables or columns: a role either may or may not `SELECT` from a table. This is insufficient when multiple distinct users share a single database login role, as is the case in this project's design, where every doctor authenticates to the database as `doctor_role`. PostgreSQL's Row-Level Security (RLS) feature, introduced in PostgreSQL 9.5, allows a table owner to attach **policies** that are evaluated per row at query time, filtering which rows a given role may see or modify based on arbitrary SQL conditions — including session-local variables set by the application after authentication. This project uses RLS specifically to ensure that two different doctors, despite sharing the same `doctor_role` login, each see only their own patients' records.

3. System Design

3.1 Functional Overview

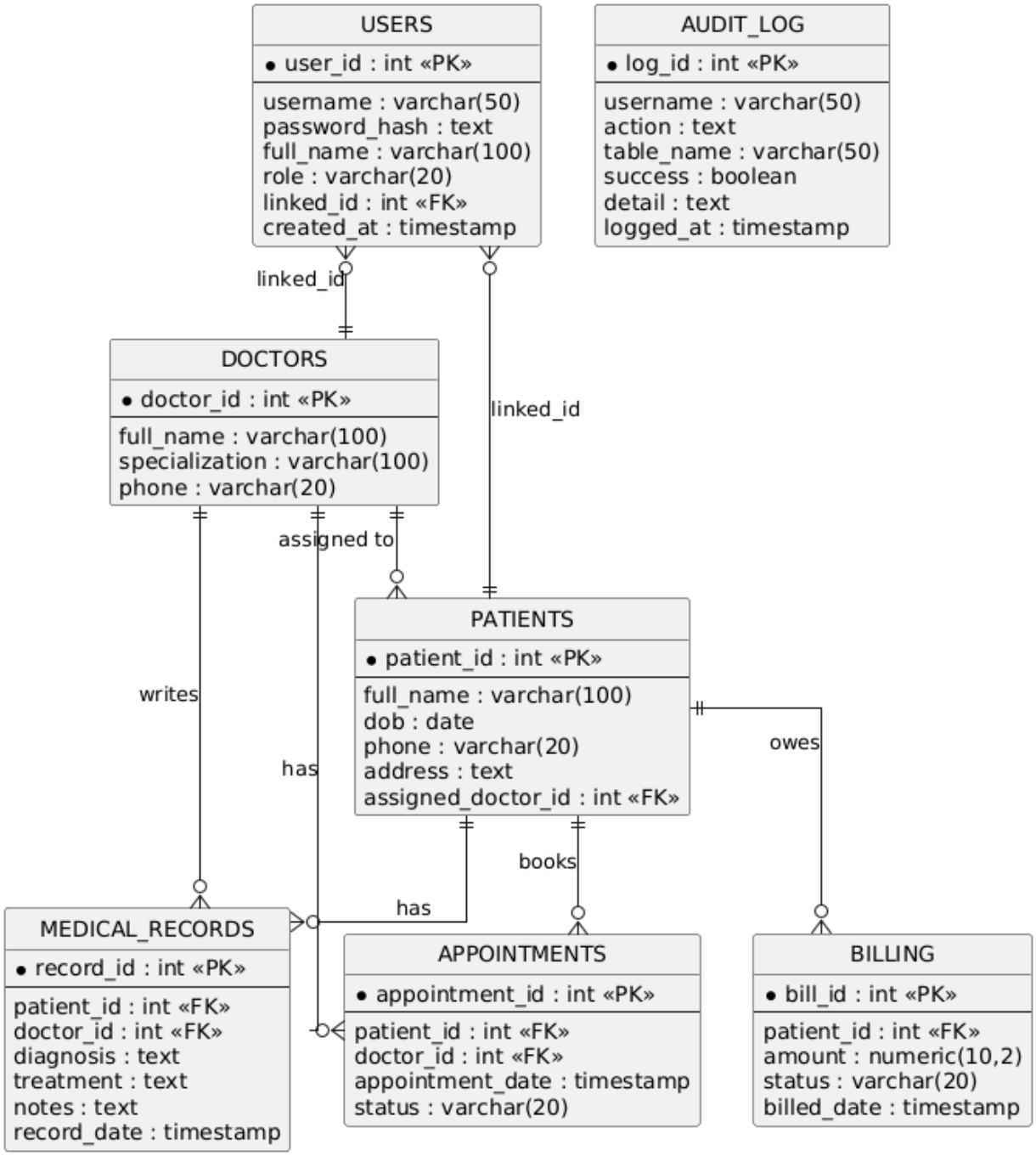
The system models a hospital named "CityCare Hospital" with the following user types, each mapped to both an application-level role (stored in the `users` table) and a distinct PostgreSQL database role in the secure implementation:

- **Administrator** — full system access, for IT/system management purposes.
- **Doctor** — manages and reviews medical records and appointments for their own assigned patients.
- **Nurse** — reviews and updates medical records and patient information hospital-wide.
- **Receptionist** — manages patient registration, contact information, and appointment scheduling; has no clinical or financial data access.
- **Billing Clerk** — manages billing records; has no clinical data access.

3.2 Entity-Relationship Design

The database schema consists of seven tables:

- **users** — application login accounts: username, bcrypt password hash, full name, role, and an optional link to a doctor or patient record.
- **doctors** — doctor identity and specialization.
- **patients** — patient identity, contact information, and the doctor assigned to their care.
- **appointments** — scheduled appointments linking a patient and a doctor.
- **medical_records** — the most sensitive table: diagnosis, treatment, and clinical notes, linked to both a patient and the treating doctor.
- **billing** — financial records per patient: amount and payment status.
- **audit_log** — a record of access attempts, including denied attempts, used as evidence of database-level enforcement.



Key relationships: `patients.assigned_doctor_id` references `doctors.doctor_id`; `appointments` and `medical_records` each reference both `patients.patient_id` and `doctors.doctor_id`; `billing.patient_id` references `patients.patient_id`; and `users.linked_id` optionally references either a doctor or a patient record depending on the user's role.

3.3 Role-Privilege Matrix

The following matrix defines the intended access boundary for each role, which is enforced in the secure implementation through a combination of table-level `GRANT`/`REVOKE` and row-level RLS policies:

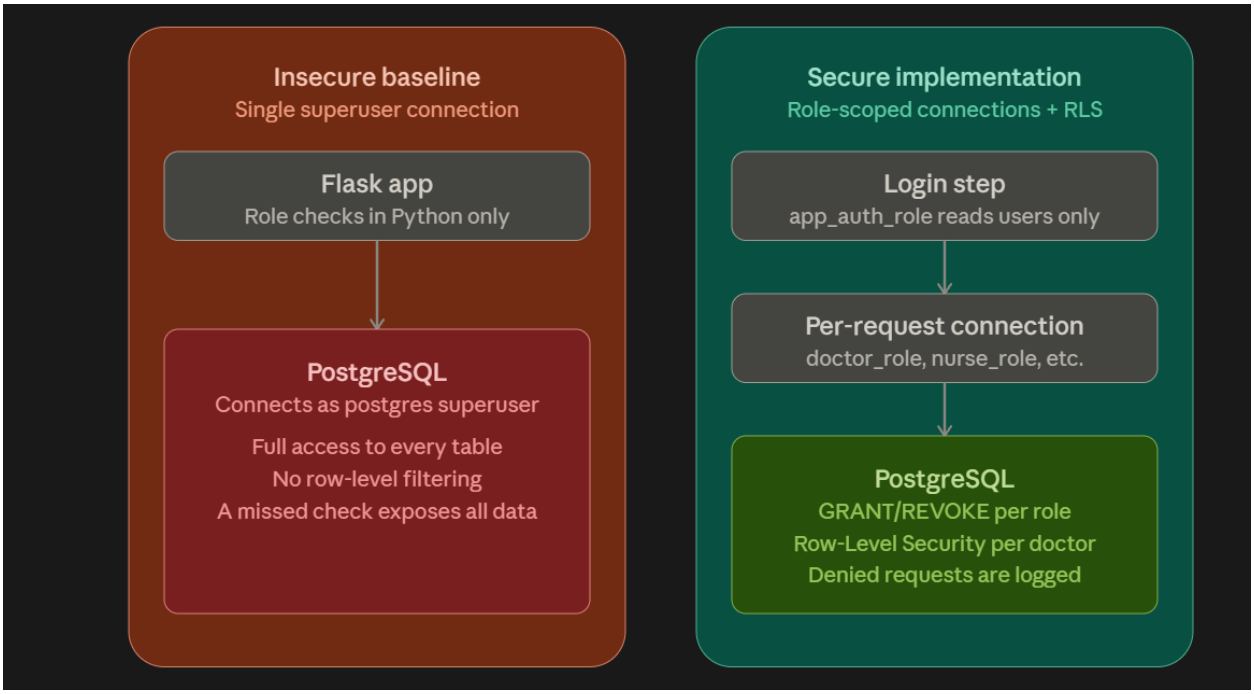
Role	Patients	Appointments	Medical Records	Billing
admin	Full access	Full access	Full access	Full access
doctor	Own assigned patients only (RLS-filtered)	Own appointments only (RLS-filtered)	Own patients' records only (RLS-filtered)	No access (table-level deny)
nurse	All patients (read)	All appointments (read)	All records (read/write)	No access (table-level deny)
receptionist	All patients (read/write)	All appointments (read/write)	No access (table-level deny)	No access (table-level deny)
billing_clerk	All patients (read only, for labeling bills)	No access (table-level deny)	No access (table-level deny)	Full access (read/write)

This matrix was deliberately designed so that the receptionist role — the role used in the IDOR attack demonstration — has **no** database-level privilege on `medical_records` whatsoever, regardless of what the application code does or fails to check.

3.4 Architecture Comparison

Insecure baseline architecture: The Flask application maintains a single database connection configuration, authenticating to PostgreSQL as the `postgres` superuser for every request regardless of which human user is logged into the application. Access control exists solely as Python conditional statements inside each route handler.

Secure implementation architecture: The Flask application uses two distinct connection paths. First, an `app_auth_role` — a PostgreSQL role with `SELECT` privilege on the `users` table only — is used exclusively to verify a submitted username and password during login. Second, once a user is authenticated, every subsequent request for that user's session opens a fresh database connection authenticated as the actual PostgreSQL role corresponding to that user's type (e.g., `doctor_role`). If the authenticated user is a doctor, the application additionally sets a session-local variable, `app.current_doctor_id`, immediately after connecting; this variable is read by the Row-Level Security policies defined on `medical_records`, `patients`, and `appointments` to filter returned rows to that specific doctor's own patients.



4. Implementation

4.1 Technology Stack

- **Database:** PostgreSQL 16, using native roles, the GRANT/REVOKE privilege system, connection-limit profiles, and Row-Level Security.
- **Backend:** Python 3 with the Flask web framework.
- **Database driver:** psycopg2.
- **Password hashing:** bcrypt (salted hash, never plain text, in both versions).
- **Session management:** Flask's signed server-side session cookies.
- **Frontend:** Server-rendered HTML templates (Jinja2) styled with Bootstrap 5.

4.2 Insecure Baseline Implementation

The insecure baseline's database access layer connects using a single superuser credential for every request:

```
def get_connection():
    conn = psycopg2.connect(
        host=os.getenv("DB_HOST"),
        port=os.getenv("DB_PORT"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_SUPERUSER"),
        password=os.getenv("DB_SUPERUSER_PASSWORD"),
    )
    return conn
```

Because every request — regardless of the logged-in user's intended role — executes with full superuser privilege, any access restriction that exists is purely a property of the Python code surrounding each route, not of the database connection itself.

4.2.1 Vulnerability 1: Insecure Direct Object Reference (IDOR)

The medical record detail route verifies only that *some* user is logged in, but not that the logged-in user's role is clinically appropriate, nor that the specific record belongs to that user's patients:

```
@app.route("/medical-records/<int:record_id>")
@login_required
def medical_record_detail(record_id):
    # BUG: no check that session["role"] is doctor/nurse,
    # and no check that this doctor is assigned to this patient.
    rows = query(
        """SELECT mr.*, p.full_name AS patient_name, d.full_name AS doctor_name
           FROM medical_records mr
           JOIN patients p ON p.patient_id = mr.patient_id
           JOIN doctors d ON d.doctor_id = mr.doctor_id
           WHERE mr.record_id = %s""",
        (record_id,),
    )
    ...
```

Because `@login_required` only checks for the presence of a session, and the query itself applies no further restriction, any authenticated user — including a receptionist, who should have zero clinical data access — can retrieve any patient's diagnosis, treatment, and notes simply by changing the integer in the URL.

4.2.2 Vulnerability 2: SQL Injection

The patient search feature constructs its SQL query using Python string formatting rather than parameter binding:

```
@app.route("/search", methods=["GET", "POST"])
@login_required
def search():
    results = []
    if request.method == "POST":
        search_term = request.form["name"]
        # VULNERABLE: never do this -- string-built SQL
        sql = f"SELECT * FROM patients WHERE full_name ILIKE '%{search_term}%'"
        results = query(sql)
    ...
```

Because the user-supplied `search_term` is concatenated directly into the SQL string, an attacker can inject SQL syntax that alters the query's meaning entirely, as demonstrated in Section 5.

4.3 Secure Implementation

4.3.1 Role Creation and Connection Limits (Profiles)

Each application-level user type is mapped to a dedicated PostgreSQL login role, with a connection limit attached directly to the role — functionally equivalent to the connection-limit component of an Oracle profile:

```
CREATE ROLE admin_role        LOGIN PASSWORD 'AdminRole@123'     CONNECTION LIMIT 5;
CREATE ROLE doctor_role       LOGIN PASSWORD 'DoctorRole@123'    CONNECTION LIMIT 10;
CREATE ROLE nurse_role        LOGIN PASSWORD 'NurseRole@123'     CONNECTION LIMIT 10;
CREATE ROLE receptionist_role LOGIN PASSWORD 'ReceptionRole@123' CONNECTION LIMIT 10;
CREATE ROLE billing_role      LOGIN PASSWORD 'BillingRole@123'   CONNECTION LIMIT 5;
```

A dedicated minimal-privilege role is used exclusively for the login step, so that even the act of checking a password does not require broad database access:

```
CREATE ROLE app_auth_role LOGIN PASSWORD 'AuthRole@123' CONNECTION LIMIT 20;
GRANT SELECT ON users TO app_auth_role;
GRANT SELECT, INSERT ON audit_log TO app_auth_role;
```

4.3.2 Privilege Grants (Least Privilege)

The default PostgreSQL behavior of implicitly granting read access to `PUBLIC` is explicitly revoked, after which privileges are granted back only as needed per role:

```
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM PUBLIC;

-- Receptionist: patient/appointment management only.
-- No grant exists on medical_records or billing for this role --
-- Postgres denies by default in the absence of a grant.
GRANT SELECT, INSERT, UPDATE ON patients TO receptionist_role;
GRANT SELECT, INSERT, UPDATE ON appointments TO receptionist_role;
GRANT SELECT ON doctors TO receptionist_role;

-- Doctor: clinical read/write, explicitly no billing access.
GRANT SELECT ON patients, doctors, appointments TO doctor_role;
GRANT SELECT, INSERT, UPDATE ON medical_records TO doctor_role;

-- Billing clerk: financial read/write, explicitly no clinical access.
GRANT SELECT ON patients TO billing_role;
GRANT SELECT, INSERT, UPDATE ON billing TO billing_role;
```

This directly neutralizes the IDOR vulnerability at its root cause: even though the secure implementation's `/medical-records/<id>` route is written with the same permissive Python logic as the insecure baseline (no role check in the application code), the underlying database connection for a receptionist simply has no `SELECT` privilege on `medical_records`, so the query raises a `psycopg2.errors.InsufficientPrivilege` exception regardless of what the application intended.

4.3.3 Row-Level Security Policies

Because multiple doctors share the single `doctor_role` login, table-level grants alone cannot distinguish between them. Row-Level Security solves this by filtering individual rows based on a session variable set immediately after login:

```
ALTER TABLE medical_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE patients        ENABLE ROW LEVEL SECURITY;
ALTER TABLE appointments    ENABLE ROW LEVEL SECURITY;

CREATE POLICY doctor_own_records ON medical_records
    FOR ALL
    TO doctor_role
    USING (doctor_id = NULLIF(current_setting('app.current_doctor_id', true),
''')::int);

CREATE POLICY doctor_own_patients ON patients
    FOR SELECT
    TO doctor_role
    USING (assigned_doctor_id = NULLIF(current_setting('app.current_doctor_id',
true), ''')::int);

ALTER ROLE admin_role BYPASSRLS;
```

The application sets the session variable immediately after opening a connection for a doctor:

```
def get_role_connection(app_role, doctor_id=None):
    db_user, db_password = ROLE_CREDENTIALS[app_role]
    conn = psycopg2.connect(host=DB_HOST, port=DB_PORT, dbname=DB_NAME,
                            user=db_user, password=db_password)
    if app_role == "doctor" and doctor_id is not None:
        cur = conn.cursor()
        cur.execute("SET app.current_doctor_id = %s", (str(int(doctor_id)),))
        cur.close()
    return conn
```

Because `doctor_id` originates from the trusted `users` table rather than raw user input, and is explicitly cast to `int` before use, this mechanism introduces no injection risk of its own.

4.3.4 Parameterized Queries

The secure implementation's search route eliminates the SQL Injection vulnerability by passing user input as a bound parameter rather than concatenating it into the query string:

```
results = role_query(
    session["role"],
    "SELECT * FROM patients WHERE full_name ILIKE %s ORDER BY patient_id",
    (f"%{search_term}%",),
    doctor_id=current_doctor_id(),
)
```

Here, `psycopg2` sends the query text and the parameter value to PostgreSQL separately; the database never interprets the parameter's contents as part of the SQL syntax, regardless of what characters it contains.

4.3.5 Audit Logging

Every access attempt denied by the database is explicitly recorded by the secure application:

```
except psycopg2.errors.InsufficientPrivilege:
    log_event(session["username"], "SELECT", "medical_records", False,
              f"Denied: tried to view record_id={record_id}")
    flash("Access denied by the database: your role cannot view medical records.",
"danger")
    return redirect(url_for("dashboard"))
```

This provides a queryable, timestamped record of exactly who attempted exactly what action, and whether it succeeded — supporting monitoring and non-repudiation, consistent with Module 5 of the course syllabus.

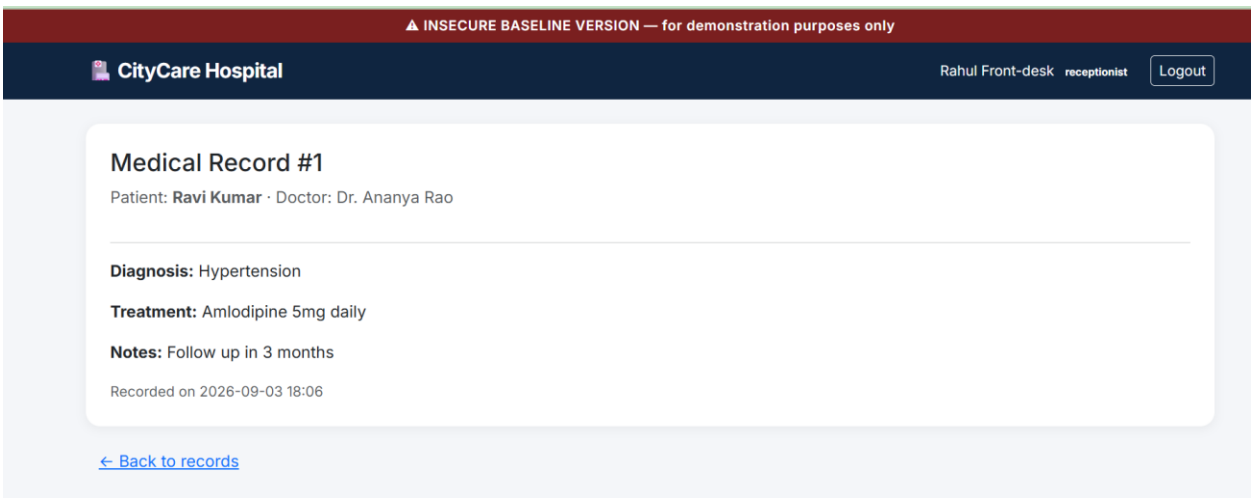
5. Attacks and Results

Each attack below was performed identically — same user account, same URL or input — against both the insecure baseline and the secure implementation, to isolate the effect of the database-level controls as the only variable.

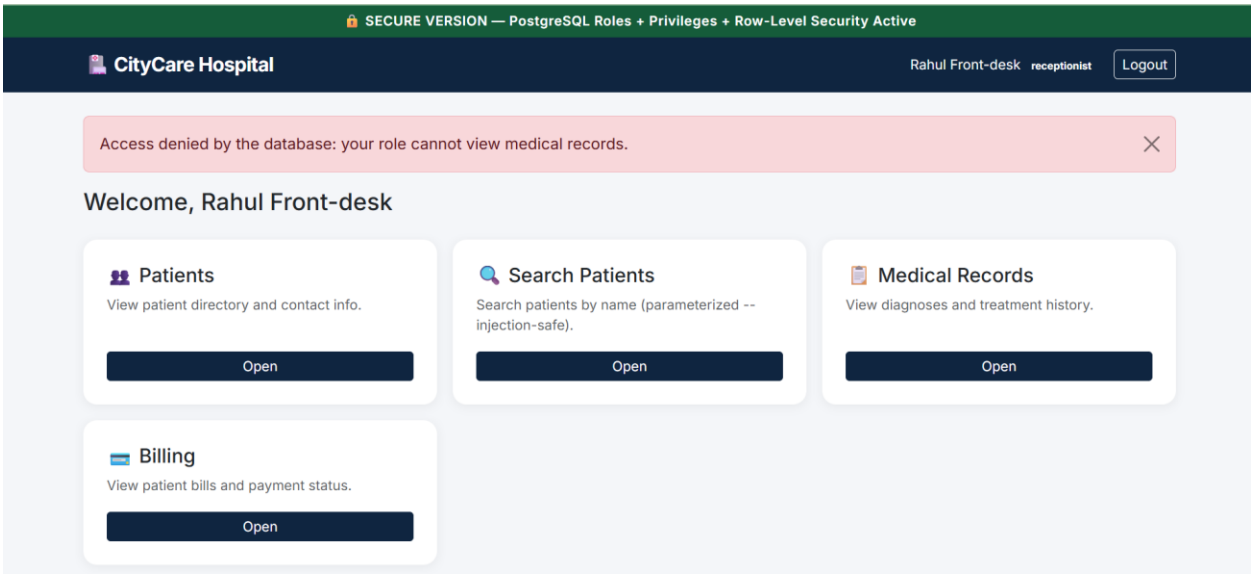
5.1 Attack 1: Insecure Direct Object Reference (IDOR)

Method: Logged in as `frontdesk1` (receptionist role), then navigated directly to `/medical-records/1` by typing the URL, without using any link provided by the application's own navigation.

Insecure baseline result: The full medical record was returned and rendered, including the patient's diagnosis ("Hypertension"), prescribed treatment, and the treating doctor's private notes — despite the receptionist role having no legitimate clinical need to view this information, and despite the application's own dashboard not even presenting a link to this page for this role.



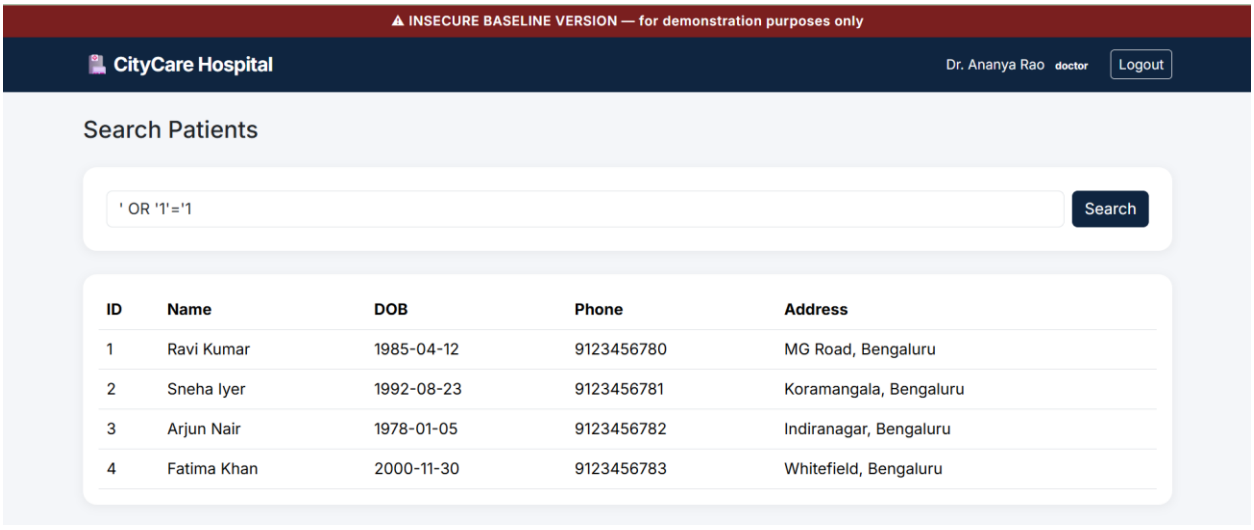
Secure implementation result: The identical request, using the identical account and URL, resulted in an `InsufficientPrivilege` exception from PostgreSQL, caught by the application and displayed as: "Access denied by the database: your role cannot view medical records." The attempt was additionally written to the `audit_log` table.



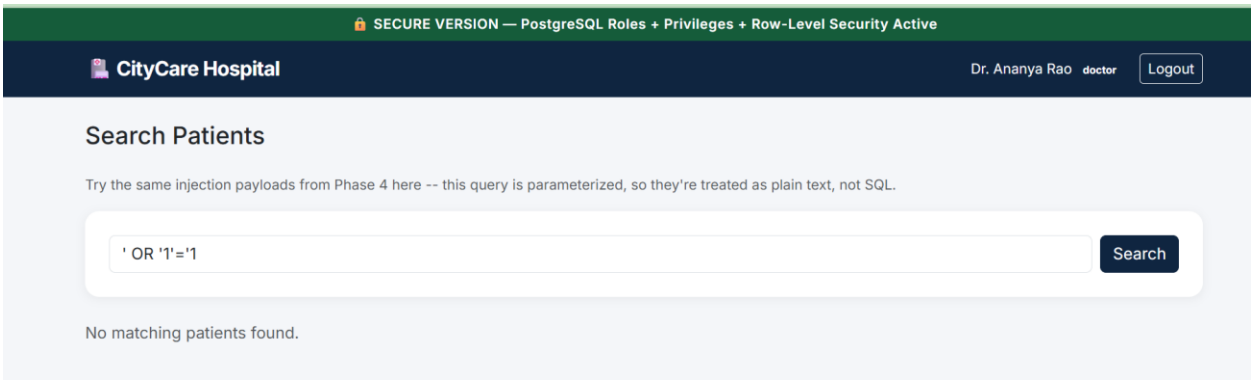
5.2 Attack 2: SQL Injection — Filter Bypass

Method: Logged in as `drrao` (doctor role), navigated to the patient search page, and submitted the payload `' OR '1'='1` in the name field.

Insecure baseline result: All four patients in the database were returned, despite the search term matching no actual patient name. The injected `OR '1'='1'` clause is always true, converting the intended filter into one that matches every row.



Secure implementation result: Because the query is parameterized, the entire string `' OR '1'='1` was treated as literal search text. No patient matched, and the page correctly reported "No matching patients found."

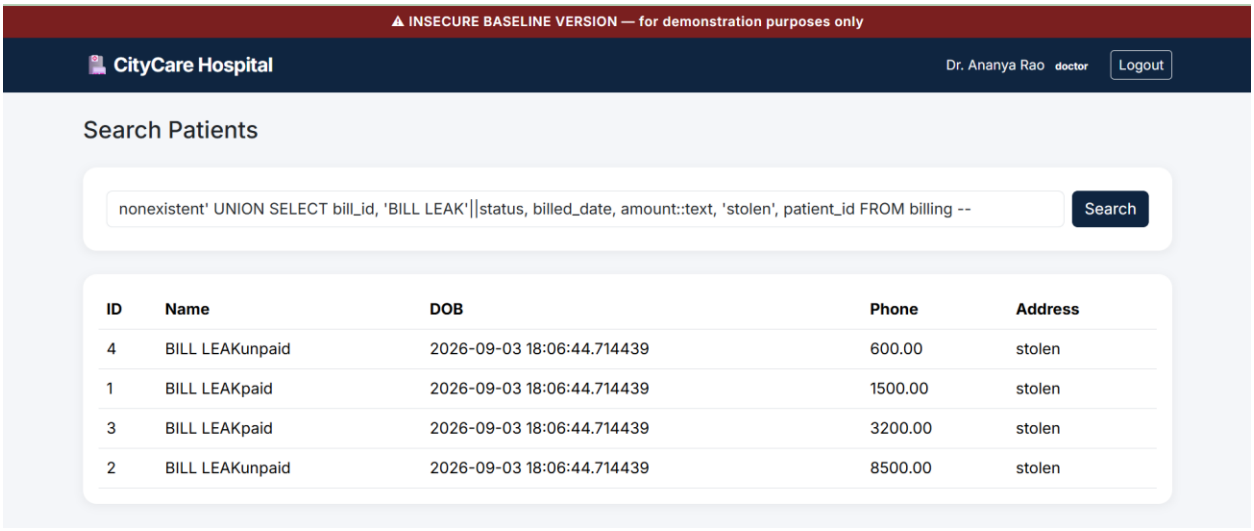


5.3 Attack 3: UNION-Based SQL Injection — Cross-Table Data Exfiltration

Method: Using the same vulnerable search field, the following payload was submitted, engineered to match the column count and compatible data types of the `patients` table's `SELECT` statement:

```
nonexistent' UNION SELECT bill_id, 'BILL LEAK'||status, billed_date,
amount::text, 'stolen', patient_id FROM billing --
```

Insecure baseline result: The patient search results table — normally displaying patient ID, name, date of birth, phone, and address — instead displayed rows from the entirely unrelated `billing` table: bill identifiers, payment status, billed dates, and amounts, all exfiltrated through a page whose only intended function was searching patient names by a doctor account that has no legitimate reason to view billing data at all.

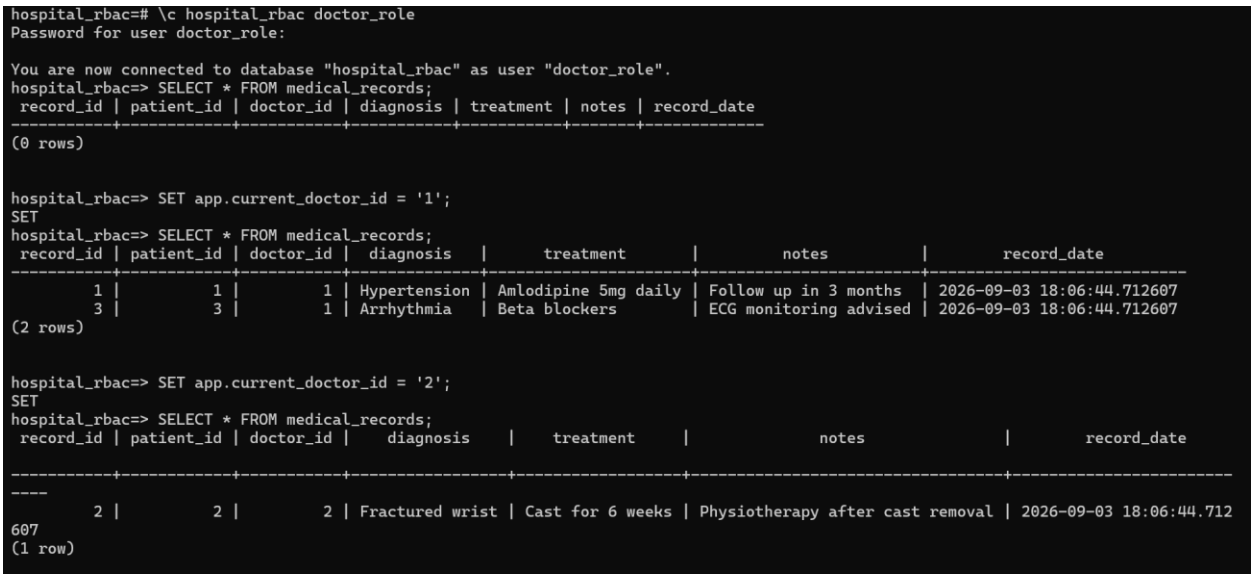


Secure implementation result: The parameterized query again rendered the injected SQL inert, returning zero matches, and furthermore the `doctor_role` database login has no `GRANT` on the `billing` table whatsoever — so even had the injection somehow succeeded syntactically, the underlying role would still have been denied access to that table's data at the privilege level, providing a second, independent layer of defense against this specific attack.

5.4 Attack 4: Row-Level Isolation Verification

Method: Connected directly to PostgreSQL as `doctor_role` (bypassing the application entirely) and queried `medical_records` after setting the session variable `app.current_doctor_id` to different values corresponding to two different doctors in the seed data.

Result: With `app.current_doctor_id` unset, zero rows were returned (safe default-deny behavior). With the variable set to doctor ID 1 (Dr. Ananya Rao), exactly the two records belonging to that doctor's patients were returned. With the variable set to doctor ID 2 (Dr. Vikram Shah), exactly the one record belonging to that doctor's patient was returned — a completely different, correctly-scoped result, using the identical login role and password throughout.



```
hospital_rbac=# \c hospital_rbac doctor_role
Password for user doctor_role:

You are now connected to database "hospital_rbac" as user "doctor_role".
hospital_rbac=> SELECT * FROM medical_records;
 record_id | patient_id | doctor_id | diagnosis | treatment | notes | record_date
-----------+------------+-----------+-----------+-----------+-------+-------------
(0 rows)


hospital_rbac=> SET app.current_doctor_id = '1';
SET
hospital_rbac=> SELECT * FROM medical_records;
 record_id | patient_id | doctor_id |  diagnosis   |      treatment       |         notes          |        record_date
-----------+------------+-----------+--------------+----------------------+------------------------+----------------------------
         1 |          1 |         1 | Hypertension | Amlodipine 5mg daily | Follow up in 3 months  | 2026-09-03 18:06:44.712607
         3 |          3 |         1 | Arrhythmia   | Beta blockers        | ECG monitoring advised | 2026-09-03 18:06:44.712607
(2 rows)


hospital_rbac=> SET app.current_doctor_id = '2';
SET
hospital_rbac=> SELECT * FROM medical_records;
 record_id | patient_id | doctor_id |    diagnosis    |    treatment     |             notes              |        record_date

-----------+------------+-----------+-----------------+------------------+--------------------------------+------------------------
----
         2 |          2 |         2 | Fractured wrist | Cast for 6 weeks | Physiotherapy after cast removal | 2026-09-03 18:06:44.712
607
(1 row)
```

This confirms that Row-Level Security successfully distinguishes between individual users who share a single database login role, which table-level `GRANT`/`REVOKE` alone cannot achieve.

5.5 Audit Log Evidence

Querying the `audit_log` table after the above attempts against the secure implementation produced the following record (reproduced from the actual test run):

log_id	username	action	table_name	success	detail
1	frontdesk1	SELECT	medical_records	false	Denied: tried to view record_id=1
2	frontdesk1	SELECT	medical_records	false	Denied: tried to list all medical records
3	frontdesk1	SELECT	medical_records	false	Denied: tried to list all medical records
4	billing1	SELECT	medical_records	false	Denied: tried to list all medical records

```
hospital_rbac=> \c hospital_rbac postgres
Password for user postgres:

You are now connected to database "hospital_rbac" as user "postgres".
hospital_rbac=# SELECT log_id, username, action, table_name, success, detail, logged_at
hospital_rbac-# FROM audit_log
hospital_rbac-# ORDER BY log_id DESC
hospital_rbac-# LIMIT 10;
 log_id |  username  | action |   table_name    | success |                 detail                 |         logged_at
--------+------------+--------+-----------------+---------+----------------------------------------+----------------------------
      5 | frontdesk1 | SELECT | medical_records | f       | Denied: tried to view record_id=1      | 2026-10-23 10:24:21.073823
      4 | billing1   | SELECT | medical_records | f       | Denied: tried to list all medical records | 2026-10-23 02:49:12.847179
      3 | frontdesk1 | SELECT | medical_records | f       | Denied: tried to list all medical records | 2026-10-23 02:47:38.521053
      2 | frontdesk1 | SELECT | medical_records | f       | Denied: tried to list all medical records | 2026-10-23 02:47:34.275897
      1 | frontdesk1 | SELECT | medical_records | f       | Denied: tried to view record_id=1      | 2026-10-23 02:47:04.320672
(5 rows)
```

This provides a timestamped, queryable, non-repudiable record of every denied access attempt — evidence that the database-level defense is not only effective but also observable and auditable, directly addressing the monitoring concerns raised in Module 5 of the course syllabus (securing database-to-database communications and monitoring usage).

5.6 Consolidated Results Table

#	Attack	Insecure App Result	Secure App Result
1	Receptionist opens `/medical-records/1` directly	Full diagnosis exposed	"Access denied by the database"
2	Doctor views all medical records via list page	Sees every patient's records, not just their own	RLS automatically filters to only their own patients

#	Attack	Insecure App Result	Secure App Result
3	`' OR '1'='1` in search box	Returns all 4 patients (filter bypassed)	Searched literally as text, zero results
4	UNION-based injection to read billing table	Billing data leaked through patient search	Parameterized query — no injection possible; role also lacks billing privilege

6. Conclusion

This project set out to answer a specific question: does enforcing access control at the database layer — in addition to the application layer — provide a meaningful, demonstrable security benefit over application-only enforcement? The results across four distinct attack scenarios support an unambiguous conclusion: it does.

In the insecure baseline, every attack succeeded completely, because the single point of enforcement — Python conditional logic scattered across route handlers — either contained a bug (the IDOR vulnerability, where a role check was simply omitted) or was itself the attack surface (the SQL Injection vulnerability, where untrusted input was concatenated into executable SQL). Critically, the underlying database connection placed no limits of its own: because the application connected as a PostgreSQL superuser, any successful bypass of the application layer immediately granted unrestricted access to every table in the system.

In the secure implementation, the same underlying route code — in the case of the medical record detail page, code that is structurally identical to the insecure version, including the same absence of an explicit Python role check — produced an entirely different outcome, because the database connection itself was now scoped to a role with only the privileges that role should legitimately have. This is the practical meaning of defense-in-depth: a flaw in one layer (the application's forgotten check) did not result in total compromise, because an independent layer (the database's privilege system) still enforced the correct boundary.

Row-Level Security addressed a subtlety that simple table-level grants cannot: when multiple distinct human users (multiple doctors) share a single database login role for practical reasons, table-level privileges alone cannot distinguish between them. By filtering individual rows based on a session variable set at login time, RLS achieved per-user isolation without requiring a separate database role per individual user — a design that scales to a hospital with any number of doctors without database administration overhead growing linearly with staff size.

Finally, the audit logging mechanism demonstrated that database-level enforcement is not a silent, opaque process — denied attempts are themselves valuable security telemetry, supporting detection of in-progress attacks and providing evidence for incident response or compliance purposes.

The overall conclusion is that application-level access control, while necessary for usability and for providing clear, role-appropriate user interfaces, is not sufficient on its own for protecting sensitive

data. Database-level enforcement — through the principle of least privilege expressed via roles and grants, and through fine-grained row filtering via Row-Level Security — provides a second, independent, and measurably effective layer of defense that holds even when the first layer fails.

6.1 Limitations and Future Work

This project's secure implementation maps multiple individual users (e.g., all doctors) to a shared database role, distinguishing them via a session variable rather than via fully independent database logins per user. While this design was shown to be effective and scales well administratively, a future extension could explore PostgreSQL's native support for per-user roles combined with role inheritance, which would allow individual-level audit trails at the database connection level itself, rather than relying solely on the application-layer audit log. Additionally, the secure implementation's RLS policies currently cover `medical_records`, `patients`, and `appointments`; a complete production system would extend equivalent row-level policies to the `billing` table (e.g., restricting a billing clerk to patients within their assigned region or department, if such a structure existed) and would integrate automated alerting on repeated denied access attempts from a single account, which could indicate an active attack in progress.

7. References

1. Hassan A. Afyouni, *Database Security and Auditing*, India Edition, CENGAGE Learning, 2009.
2. Ron Ben Natan, *Implementing Database Security and Auditing*, Elsevier, Indian Reprint, 2006.
3. Dafydd Stuttard and Marcus Pinto, *The Web Application Hacker's Handbook*, Second Edition, Wiley Publishing, 2011.
4. Castano, S. and Fugini, M., *Database Security*, Addison Wesley, ACM, 2004.
5. Fernandez, E., Summers, R., and Wood, C., *Database Security and Integrity*, Addison Wesley, 2012.
6. OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, available at: https://owasp.org/Top10/
7. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation — Chapter 5.9: Row Security Policies*, available at: https://www.postgresql.org/docs/current/ddl-rowsecurity.html
8. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation — Chapter 22: Database Roles*, available at: https://www.postgresql.org/docs/current/user-manag.html